HOUDRÉ Enola

DISCALA-PORRO Julian

Projet : Analyse Harmonique Pour L'Ingénieur

Préambule : Ce projet a pour objectif, dans une première partie de comprendre et appréhender numériquement les transformées de fourrier grâce à python, puis dans une seconde partie, analyser un signal sonore par une transformée de Fourrier. On nous donne quelques fonctions qui sont les suivantes :

1. x(t) = sin(20πt)
2. x(t) = cos(20πt + 3)
3. x(t) = 2 cos(10πt) + 7 cos(2πt)
4. x(t) = |sin(πt)|
5. x(t) = sin(50t)
6. x(t) = sin(202πt)

Les données qui nous seront utiles sont écrit dans le programme suivant :

```
import numpy as np
import matplotlib.pyplot as plt
import soundfile as sf


N=1000
T=10
dt=T/N
df=1/T

t=np.linspace(0,T,N,endpoint=False)
```

PARTIE 1 :

Comme expliqué précédemment, cette partie va nous permettre de comprendre comment fonctionne les signaux, grâce aux transformées de Fourrier.

1.Dans cette première question on tracera d'abord les 6 fonctions dans le temporelle grâce au programme suivant.

```
x1=np.sin(20*np.pi*t)
x2=np.cos(20*np.pi*t+3)
x3=2*np.cos(10*np.pi*t)+7*np.cos(2*np.pi*t)
x4=np.abs(np.sin(np.pi*t))
x5=np.sin(50*t)
x6=np.sin(202*np.pi*t)

plt.figure()
plt.plot(t,x1)
plt.ylabel('X1')

plt.figure()
plt.plot(t,x2)
plt.ylabel('X2')

plt.figure()
plt.plot(t,x3)
plt.ylabel('X3')

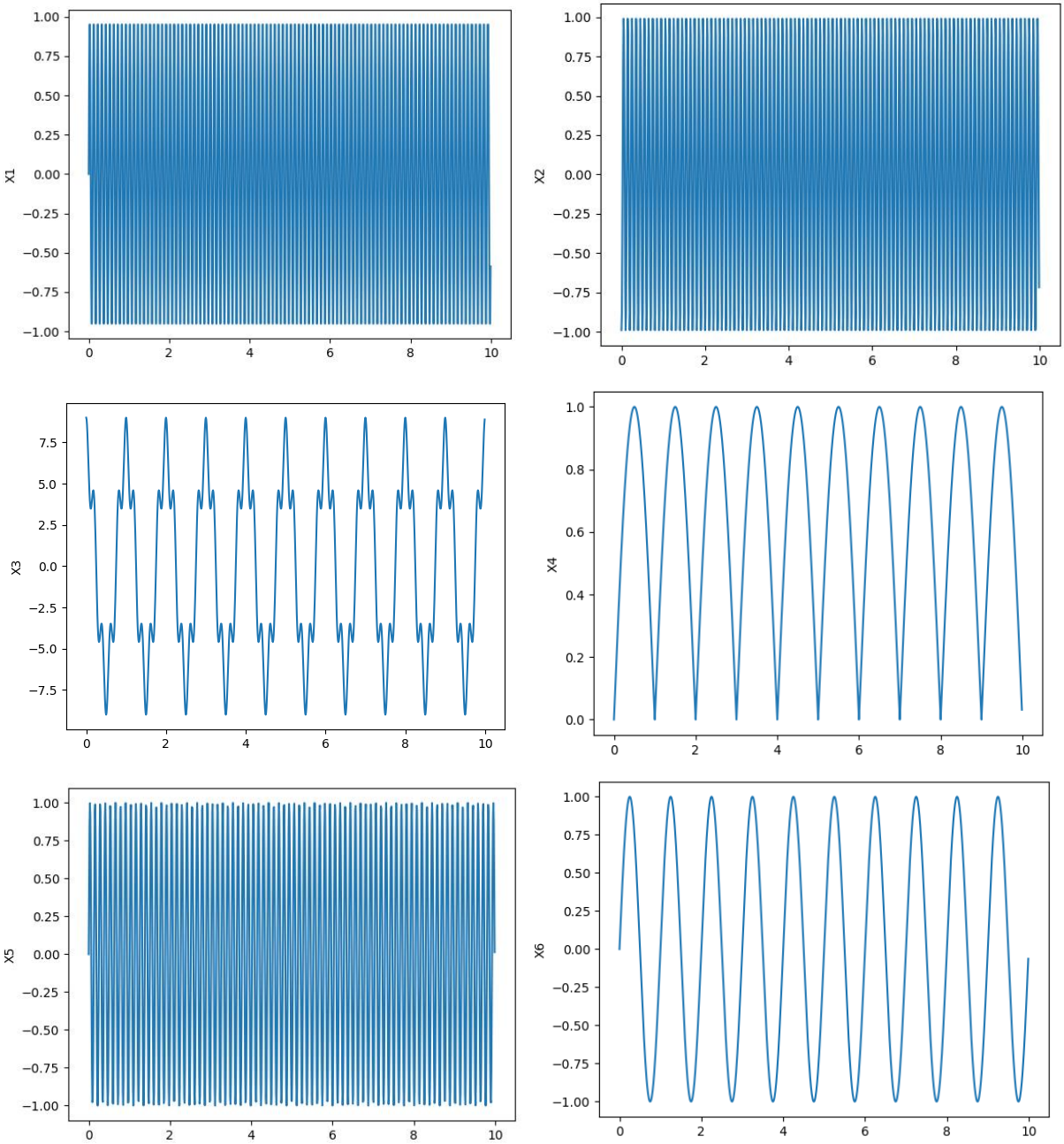
plt.figure()
plt.plot(t,x4)
plt.ylabel('X4')

plt.figure()
plt.plot(t,x5)
plt.ylabel('X5')

plt.figure()
plt.plot(t,x6)
plt.ylabel('X6')

plt.show()
```

Ce programme nous donne les courbes suivantes :



on nous demande aussi de tracer le module de nos fonctions (en bleu) dans le domaine fréquentiel auquel on rajoute la partie Imaginaire (en vert) et la partie Réel (en orange).

Le programme utilisé pour le faire est le suivant et juste après les courbes qui ont été renvoyés :

```
X1=np.fft.fft(x1)

X2=np.fft.fft(x2)


X3=np.fft.fft(x3)

X4=np.fft.fft(x4)

X5=np.fft.fft(x5)

X6=np.fft.fft(x6)

f=np.linspace(0,N*df,N,endpoint=False)


plt.figure()
plt.plot(f,np.abs(X1))
plt.plot(f,np.real(X1))
plt.plot(f,np.imag(X1))
plt.ylabel('X1')
```

```
plt.figure()
plt.plot(f,np.abs(X2))
plt.plot(f,np.real(X2))
plt.plot(f,np.imag(X2))
plt.ylabel('X2')

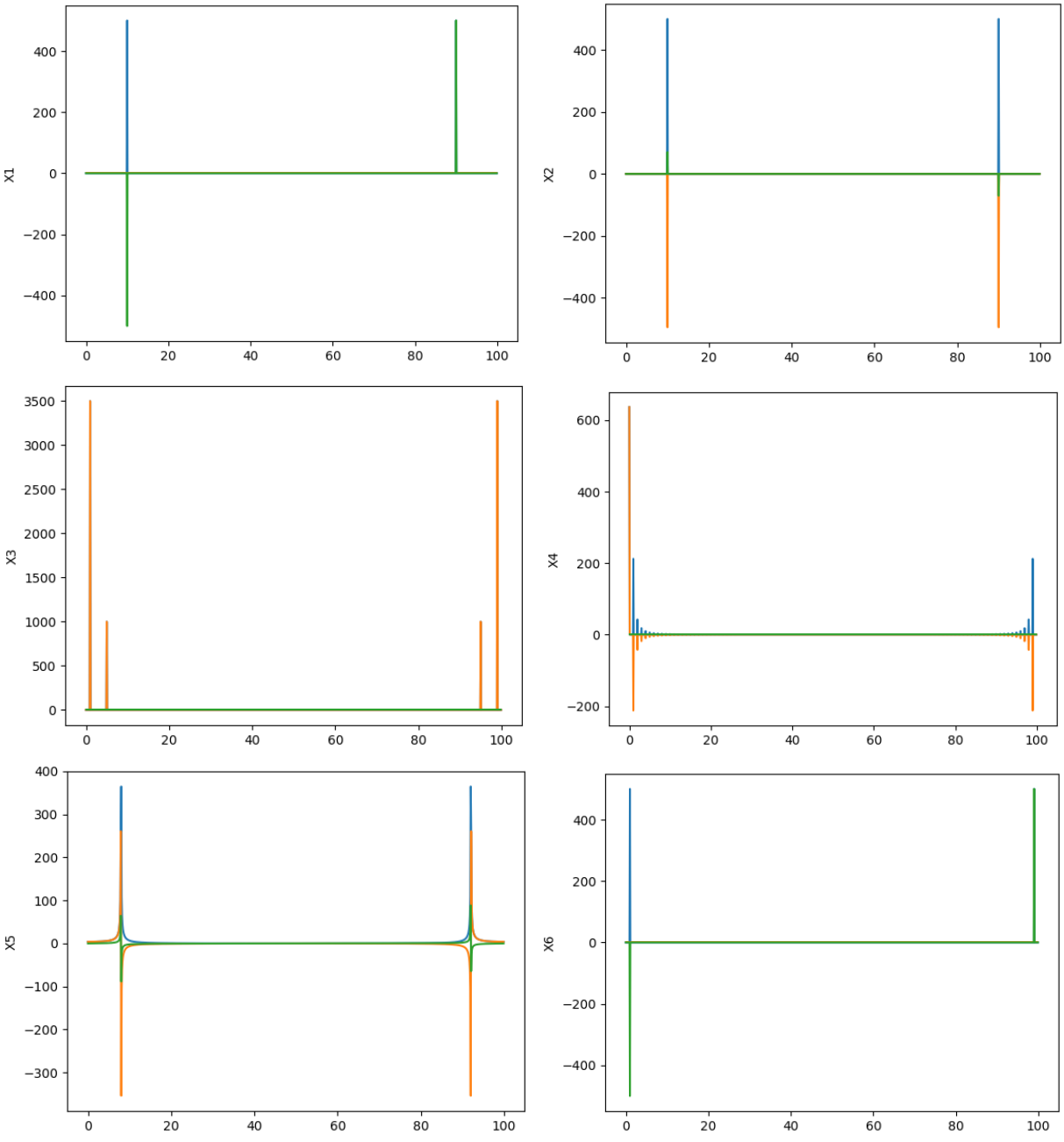

plt.figure()
plt.plot(f,np.abs(X3))
plt.plot(f,np.real(X3))
plt.plot(f,np.imag(X3))
plt.ylabel('X3')


plt.figure()
plt.plot(f,np.abs(X4))
plt.plot(f,np.real(X4))
plt.plot(f,np.imag(X4))
plt.ylabel('X4')


plt.figure()
plt.plot(f,np.abs(X5))
plt.plot(f,np.real(X5))
plt.plot(f,np.imag(X5))
plt.ylabel('X5')

plt.figure()
plt.plot(f,np.abs(X6))
plt.plot(f,np.real(X6))
plt.plot(f,np.imag(X6))
plt.ylabel('X6')
plt.show()
```

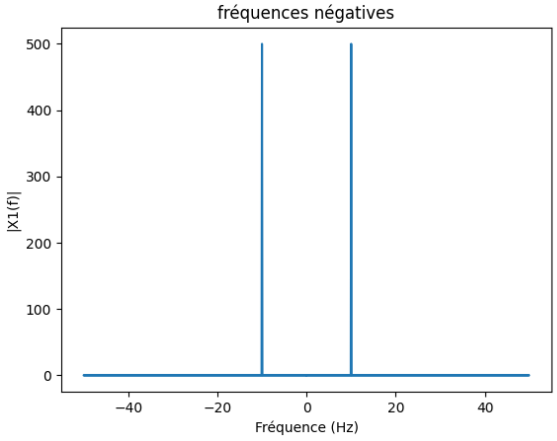
Et voici les courbe s qui ont été tracé par le programme :

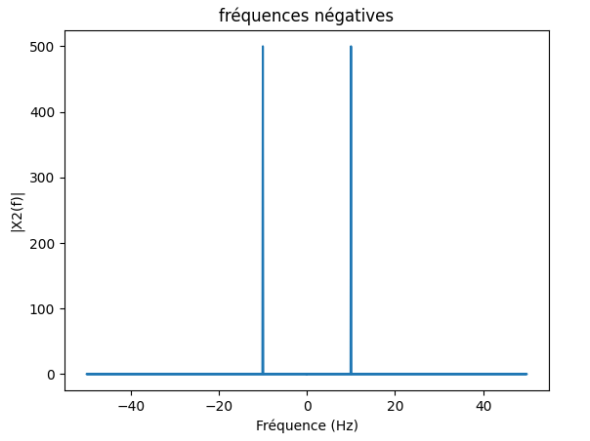


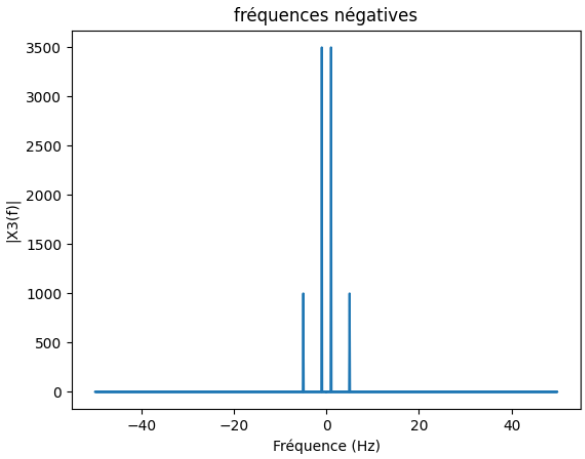
2.On remarque des valeurs nulles sur l'ensemble du spectre sauf à certaines fréquences où il y'a un « pique » prenant de bien plus grandes valeurs. C'est grâce à ces piques que l'on identifiera les fréquences de nos sinusoïdes.

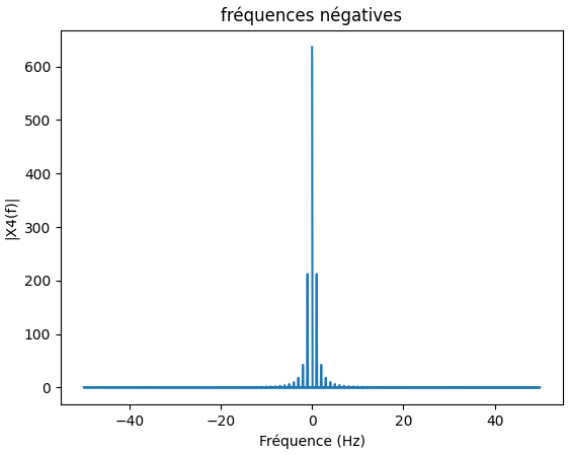
3. On remarque que les fréquences obtenus $\frac{1}{2}f_e < f_k < f_e$ peuvent être interprété comme des fréquences négatives grâce à la symétrie des transformées de Fourier. On le voit très bien graphiquement, avec un pique à gauche et un autre à droite en f_k=$f_k - f_e$.

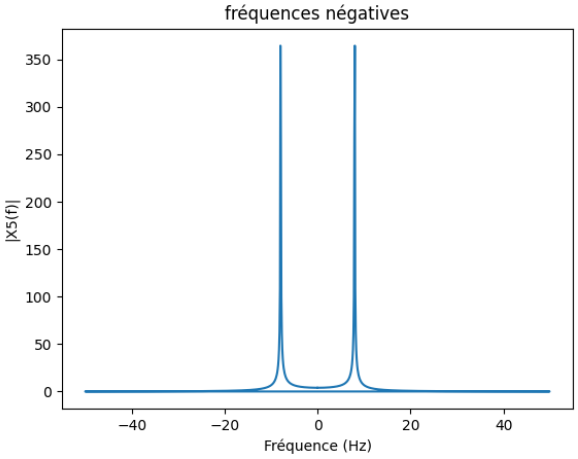
4.en prenant en compte ce point de vue on obtient les spectres suivants :

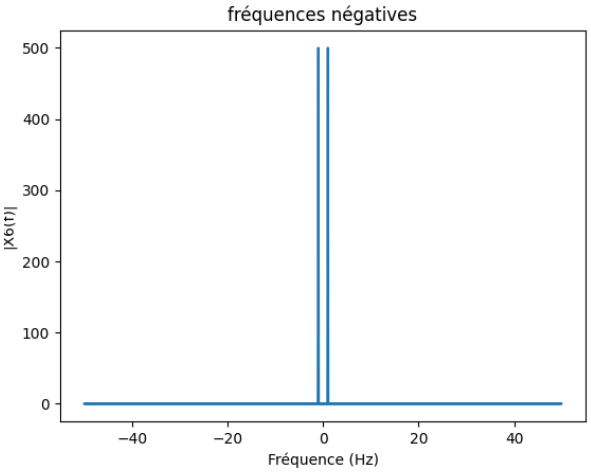












5.Dans le cas ou x(t) est réel, on obtient graphiquement la relation suivante X(-f) = X(f)

On peut même rajouter que Im(X(-f)) = -Im(X(f)). Avec la même partie réelle et des parties imaginaire opposée on obtient que X(-f) est le conjugué de x(f).

PARTIE 2 : dans cette partie l'objectif sera d'analyser le spectre d'un fichier audio.

1. Premièrement, nous allons lire un fichier audio à l'aide de la fonction scipy.io.wavfile.read. Cela nous permettra d'extraire la fréquence d'échantillonnage et les échantillons audio dans un tableau.

```
# 1.Lecture du fichier audio

fe, signal = wavfile.read('Note_mono_08.wav')  # Remplacer par le chemin du fichier audio
```

2. Ensuite on calcule la durée du fichier audio en millisecondes. Pour cela, nous utiliserons le nombre d'échantillons et la fréquence d'échantillonnage.

N=101999

La durée en millisecondes est calculée par la formule :

$$\text{durée} = \frac{N}{f_e} \times 1000$$

où N est le nombre d'échantillons.

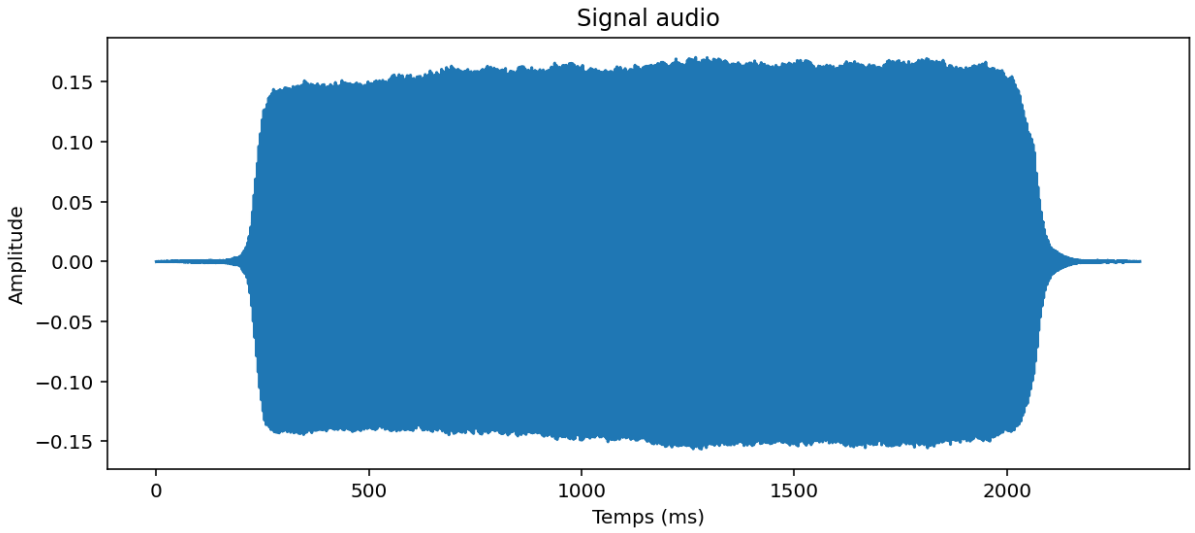
```
N = len(signal)
duree_ms = (N / fe) * 1000
print(f"La durée du fichier audio est de {duree_ms:.2f} ms")
```

On obtient : La durée du fichier audio est de 2312.90 ms

3. Puis, on trace le signal audio en fonction du temps. On utilise matplotlib pour créer un graphique qui montre comment l'amplitude du signal varie au fil du temps.

```
21
22  # 3.Répresentation du signal
23
24  N = len(signal)  # Nombre d'échantillons
25  temps = np.linspace(0, N / fe, N)
26  plt.figure(figsize=(10, 4))
27  plt.plot(temps * 1000, signal)  # Temps en millisecondes
28  plt.title('Signal audio')
29  plt.xlabel('Temps (ms)')
30  plt.ylabel('Amplitude')
31  plt.show()
32
```

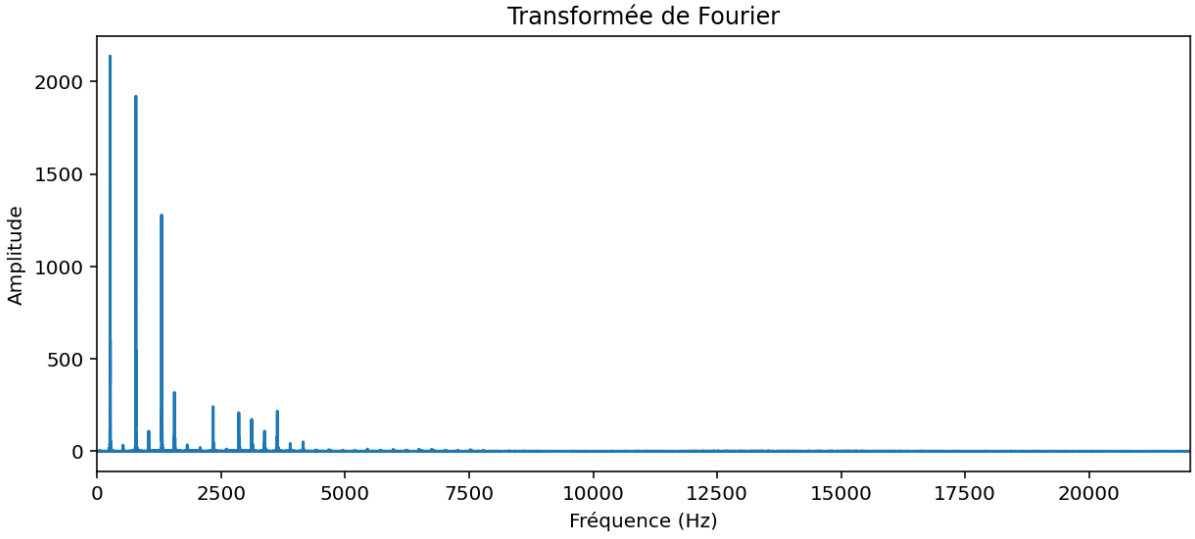
On obtient le graphique suivant :



4. On effectue une transformée de Fourier pour obtenir la représentation fréquentielle du signal. Cela nous permettra de visualiser les différentes fréquences présentes dans le son.

```
34
35  # 4.Représentation fréquentielle
36
37  transfo_fourier = fft(signal) #Transformé de Fourier
38  frequences = np.fft.fftfreq(N, 1 / fe)
39  positive_freq = frequences[:N // 2] #frequence positive
40  positive_spectrum = np.abs(transfo_fourier[:N // 2])
41  plt.figure(figsize=(10, 4))
42  plt.plot(positive_freq, positive_spectrum)
43  plt.title('Transformée de Fourier')
44  plt.xlabel('Fréquence (Hz)')
45  plt.ylabel('Amplitude')
46  plt.xlim(0, fe // 2)
47  plt.show()
48
```

On obtient le tracé suivant :



5. On identifie maintenant la fréquence fondamentale F0 ainsi que les premières harmoniques en utilisant la méthode des pics sur le spectre obtenu précédemment.

```
49
50
51  # 5.Fréquence fondamentale et harmonique
52
53  indice_f0 = np.argmax(positive_spectrum)
54  F0 = positive_freq[indice_f0]  #Fréquence fondamentale
55  harmoniques = [F0 * i for i in range(1, 6)] #extrait les 5 premiere harmonique
56  amplitudes = [positive_spectrum[np.argmin(np.abs(positive_freq - f))] for f in harmoniques]
57  for i, (f, A) in enumerate(zip(harmoniques, amplitudes)):
58      print(f"Harmonique {i+1}: Fréquence = {f:.2f} Hz, Amplitude = {A:.2f}")
59
60
61
```

Harmonique 1: Fréquence = 259.41 Hz, Amplitude = 2138.77

Harmonique 2: Fréquence = 518.83 Hz, Amplitude = 32.89

Harmonique 3: Fréquence = 778.24 Hz, Amplitude = 1495.45

Harmonique 4: Fréquence = 1037.66 Hz, Amplitude = 63.52

Harmonique 5: Fréquence = 1297.07 Hz, Amplitude = 616.56

6. Enfin, nous allons créer un fichier audio synthétique en combinant les fréquences fondamentales et leurs harmoniques identifiées. Ce son synthétique sera basé sur une somme de sinusoïdes correspondant aux différentes fréquences.

```
60
61
62  # 6.Création d'un fichier audio
63
64  duree_synth = 3 #durée du son
65  t = np.linspace(0, duree_synth, int(fe * duree_synth), endpoint=False)
66  signal_synthetique = np.zeros_like(t)
67  for i, (f, A) in enumerate(zip(harmoniques, amplitudes)):
68      signal_synthetique += A * np.cos(2 * np.pi * f * t) #création du son comme des sommes de sinusoides
69  signal_synthetique /= np.max(np.abs(signal_synthetique)) #on nomralise le signal
70  sf.write('synthetique.wav', signal_synthetique, fe) #on enregistre le son dans un fichier
71
```